

P2308R1: Template parameter initialization

Audience: CWG

S. Davis Herring <herring@lanl.gov>

Los Alamos National Laboratory

November 8, 2023

History

Since [r0](#):

- Copyedited
- Made the temporary class-type variable `constexpr`
- Clarified constraints on and selection of the template parameter object, removing the term “exemplar”

R0: issue drafting converted to a paper after initial review.

Introduction

[CWG2459](#) points out that there is no specification for how template parameters are initialized beyond “conversion to the type of the template-parameter” ([`temp.type`]/1.3) and “a converted constant expression ([`expr.const`]) of the type of the *template-parameter*” ([`temp.arg.nontype`]/2). As originally reported by Richard Smith, with template parameters of class type that have lvalue template parameter *objects* whose addresses can be examined during [non-trivial construction](#) and for which “[converted constant expression](#)” is [inadequate](#), this becomes an acute concern. This paper resolves that issue, along with [CWG2450](#) (including an extension of [Jens Maurer’s drafting](#)) and [CWG2049](#) which extend the set of potential initializers.

Approach

To avoid address-based paradoxes, template arguments for a template parameter of class type `C` are used to first initialize a temporary of that type. No restrictions are imposed on the conversion from a template argument to a constructor parameter, since `explicit` and list-initialization may already be used to limit conversions in a similar fashion. Each temporary is used to copy-initialize the template parameter object to which it is (to be) template-argument-equivalent; the initialization is required to produce a template-argument-equivalent value. The multiple initializations of the template parameter object are (required to be) all equivalent and produce no side effects, so it is unobservable which happen. Using the same copy constructor for the temporary (when the template argument is a template parameter object) and for initializing the template parameter object guarantees that the same value will be selected, as required for deduction, specialization, and the notion of the current specialization to make sense.

Wording

Relative to N4928.

#[dcl.dcl]

#[dcl.fct.default]

Change paragraph 3:

- A default argument shall be specified only in the *parameter-declaration-clause* of a function declaration or *lambda-declarator* or in a *template-parameter* ([temp.param]); ~~in the latter case, the initializer-clause shall be an assignment-expression~~. [...]

#[dcl.init.list]

Insert before bullet (1.6):

- as a template argument ([temp.arg.nontype])

#[temp]

#[temp.param]

Change paragraph 8:

- An *id-expression* naming a non-type *template-parameter* of class type τ denotes a static storage duration object of type `const  $\tau$` , known as a *template parameter object*, ~~whose value~~ which is ~~that of~~ template-argument-equivalent ([temp.type]) to the corresponding template argument after it has been converted to the type of the *template-parameter* ([temp.arg.nontype]). No two template parameter objects are template-argument-equivalent. ~~All such template parameters in the program of the same type with the same value denote the same template parameter object. A template parameter object shall have constant destruction ([expr.const]).~~

[...]

Replace all appearances of “*template-argument*” in paragraphs 12–16 with “template argument”.

#[temp.names]

Change the grammar in paragraph 1:

- [...]

template-argument:

constant-expression

type-id

id-expression

braced-init-list

#[temp.arg]

#[temp.arg.general]

Change paragraph 4:

- [...]

For a ~~template-argument~~ template argument that is a class type or a class template, the template definition has no special access rights to the members of the ~~template-argument~~ template argument.

[Example:

[...]

— end example]

Change paragraph 5:

- When template argument packs or default ~~template-argument~~ template arguments are used, a *template-argument* list can be empty. [...]

Change paragraph 7:

- If the use of a ~~template-argument~~ template argument gives rise to an ill-formed construct in the instantiation of a template specialization, the program is ill-formed.

#[temp.arg.nontype]

Change paragraph 1:

- If the type τ of a *template-parameter* ([temp.param]) contains a placeholder type ([dcl.spec.auto]) or a placeholder for a deduced class type ([dcl.type.class.deduct]), the type of the parameter is the type deduced for the variable x in the invented declaration

$\tau \ x =$ ~~template-argument~~ E ;

where E is the template argument provided for the parameter.

[Note: E is a *template-argument* or (for a default template argument) an *initializer-clause*. — end note]

If a deduced parameter type is not permitted for a *template-parameter* declaration ([temp.param]), the program is ill-formed.

Change paragraph 2:

- ~~A *template-argument* for~~ The value of a non-type *template-parameter* P of (possibly deduced) type T is determined from its template argument A as follows. If T is not a class type and A is not a *braced-init-list*, A shall be a converted constant expression ([expr.const]) of ~~the type of the *template-parameter* T ; the value of P is A (as converted).~~

~~[Note: If the *template-argument* is an overload set (or the address of such, including forming a pointer-to-member), the matching function is selected from the set ([over.over]). — end note]~~

Insert before paragraph 3:

- Otherwise, a temporary variable

constexpr $T \ v = A$;

is introduced. The lifetime of v ends immediately after initializing it and any template parameter object (see below). For each such variable, the *id-expression* v is termed a *candidate initializer*.

- If T is a class type, a template parameter object ([temp.param]) exists that is constructed so as to be template-argument-equivalent to v ; P denotes that template parameter object. P is copy-initialized from an unspecified candidate initializer that is template-argument-equivalent to v . If, for the initialization from any candidate initializer,
 1. the initialization would be ill-formed, or
 2. the full-expression of an invented *init-declarator* for the initialization would not be a constant expression when interpreted as a *constant-expression* ([expr.const]), or
 3. the initialization would cause P to not be template-argument-equivalent ([temp.type]) to v ,
 the program is ill-formed.
- Otherwise, the value of P is that of v .

Change the example in paragraph 4:

- [...]

```
template<auto n> struct B { /* ... */ };
B<5> b1;                      // OK, template parameter type is int
B<'a'> b2;                     // OK, template parameter type is char
B<2.5> b3;                     // OK, template parameter type is double
B<void(0)> b4;                  // error: template parameter type cannot be void

template<int i> struct C { /* ... */ };
C<{ 42 }> c1;                  // OK

struct J1 {
    J1 *self=this;
};
B<J1{}> j1;                    // error: initialization of template parameter object
                                // is not a constant expression

struct J2 {
    J2 *self=this;
    constexpr J2() {}
    constexpr J2(const J2&) {}
};
B<J2{}> j2;                    // error: template parameter object not
                                // template-argument-equivalent to introduced temporary
```

#[temp.type]

Change bullet (1.3):

- the template parameter values determined by their corresponding non-type ~~template-arguments~~ template arguments ([temp.arg.nontype]) are template-argument-equivalent (see below) ~~after conversion to the type of the template-parameter~~, and